

GAME DESIGN - LECTURE 01

1. Course Direction & Expectations

- Focus: **Game Design + Game Programming**
- Heavy emphasis on **programming (GDScript + C later)**
- Beginner programming is **not deeply taught here**
- Recommended parallel learning:
 - Python (syntax similarity with GDScript)
 - Core computer science fundamentals

Key idea: This course builds **design understanding first** → **programming later (Module 2 onward)**

2. What is a Game (Core Definition)

A game is: **A system in which players engage in artificial conflict, defined by rules, that results in a quantifiable outcome.**

Breakdown:

1. System

- Interconnected elements (objects, mechanics, players)
- Example: Tetris → blocks + score + rules

2. Players: External agents interacting with the system

3. Artificial Conflict

- Competition, puzzles, time pressure
- Not real-life consequences

4. Rules: Define interactions and constraints

5. Quantifiable Outcome: Clear results (win/loss/draw/score/state change)

3. Game vs Activity (Critical Distinction)

Examples:

- **Tic-Tac-Toe** → **Game:** Player decisions matter, Strategy affects outcome
- **Chess** → **Game:** Deep strategy, Skill-based dominance
- **Snakes & Ladders** → **Not a Game (as per this framework)**
 - Pure chance, No meaningful decisions, Outcome not tied to player skill

Key Insight:

- **Games require meaningful player agency**
- Without decision-making → becomes an activity, not a true game

4. How to Fix Non-Games

To convert something like Snakes & Ladders into a game:

- Add **choice**, Introduce **risk vs reward decisions**
- **Example change:** Multiple ladders → player chooses path

5. Core Functions of Games

A game can serve multiple purposes:

- | | | |
|-------------------------|------------------------------|----------------------------|
| 1. Entertainment | 3. Storytelling | 5. Problem Solving |
| 2. Simulation | 4. Social Interaction | 6. Education |
| | | 7. Self-Improvement |

Example Mapping:

- The Sims → Simulation + Social + Entertainment

- Whispered World → Storytelling + Puzzle + Entertainment
- Operation Neptune → Education + Problem Solving
- Typing games → Self-improvement + Education

Key idea: Not all games prioritize entertainment — design intent varies.

6. Game Content Elements

Games typically include:

- Mechanics (rules + systems)
- Assets (visual/audio)
- Story
- Social systems
- Challenges
- Information

Training/exercises

7. Player Types (Bartle's Taxonomy)

Players are not fixed types — think **spectrum**.

4 Core Types:

- **Achievers:** Goals, scores, completion
- **Explorers:** Discovery, secrets, systems
- **Socializers:** Interaction with people
- **Killers:** Competition, domination

Reality: Most players = **mix of types**

One type may dominate behavior

8. Roles in Game Development

Game Designer

- Gameplay design
- Systems thinking
- Story, narrative, writing
- Directing player experience

Game Programmer

- Implements logic
- Works with game engine
- Integrates tools
- Debugging

Important:

- Designers decide **what**
- Programmers decide **how**

Hybrid Role: People with both skills act as **bridge between design and tech**

9. Industry Complexity

Game development is **multi-disciplinary**:

Core Areas:

- Art (2D, 3D, animation, level design)
- Sound (music, effects, voice)
- Programming
- UX/UI
- Game design

Supporting Areas:

- Marketing, Legal, Project management, HR, Psychology, Software engineering

Key idea: Games are not just code — they are **coordinated systems of multiple disciplines**

10. Software Engineering in Games

- Defines: What to build, When to build
- Ensures: Task clarity, Deadlines, Coordination

Without it → chaos

11. Games as Artistic Expression

- Games can express: Emotions, Themes, Perspectives
- Example themes: Relationships, Morality, Fear, loss, identity
- **Important shift:** Games are not just systems

They are also **experiences with meaning**

12. Core Takeaways (What to Actually Remember)

- A game = **system + rules + player interaction + conflict + measurable outcome**
- **Player choice is essential** — no choice → not a true game

- Games serve **multiple functions**, not just entertainment
- Players are **behavioral mixes**, not fixed types
- Game dev = **design + programming + many other disciplines**
- Designers shape experience, programmers implement it
- Games are both: **Technical systems, Artistic mediums**
- Complexity is normal — not something to avoid, but to understand

13. Practical Thinking Frame

Whenever you see or design a game, ask:

- Where is the **player decision**?
- What is the **conflict**?
- What are the **rules**?
- What is being **measured**?
- What is the **purpose** (function)?
- Who is the **player type**?

GAME DESIGN - LECTURE 02

1. Purpose of the Lesson

- Understand **what game genres are**
- Learn **how to classify a game into a genre**
- Learn **how genres combine to create new ones**

2. Major Game Genres (Core Overview)

This list is **not exhaustive** — genres evolve. and the below are just few popular Genres

1. Action

- Fast-paced, skill-based
- Combat, reflexes, movement
- Appeals to **achievers + killers**

2. Adventure

- Exploration, discovery
- Often open-world, crafting, upgrades
- Appeals to **explorers**

3. Casual

- Simple mechanics, short sessions
- Low cognitive load
- High repetition → addictive loops

4. Indie (Production Model, not gameplay)

- Independent studios (no publisher)
- More **creative freedom**, more risk
- Own IP, fewer constraints

6. Racing

- Speed + competition
- Modes: single-player, split-screen, online

7. RPG (Role-Playing Game)

- Character progression
- Inventory systems
- Player choices affect gameplay/story

8. Simulation

- Replicates real-world systems
- Simplified interaction, realistic behavior
- Often minimal or no story

9. Sports

- Real or fictional sports
- Competitive, multiplayer-focused

10. Strategy

- Planning, decision-making
- Control over multiple units/resources
- High-level thinking over reflexes

3. Key Insight: Genres Are Not Rigid

- A game can belong to **multiple genres simultaneously**

Examples:

- Action + Adventure → Action-Adventure
- Action + Adventure + RPG → Hybrid
- Racing + Simulation → Sim Racing
- MMO + RPG → MMORPG

4. How to Classify a Game Genre (Core Framework)

You **cannot** rely on one factor — classification is multi-dimensional.

Primary Axes:

1. Functions (Why the game exists)

- Entertainment, Problem-solving, Simulation, Social, etc.

2. Content (What's inside)

- Mechanics, Assets, Story, Systems

3. Audience (Who it attracts)

- Achievers, Explorers, Socializers, Killers

Important:

- These 3 alone are **not enough**, You need supporting context

5. Supporting Classification Factors

Use these to refine genre identification:

1. Number of Players

- Single-player → often adventure
- Multiplayer → action/MMO/social
- MMO → defined by scale

2. Graphics & Camera View

- 2D vs 3D → changes mechanics
- First-person → action/simulation
- Isometric → strategy/RPG/simulation

3. Platform

- Mobile → simpler input, casual bias
- PC → complex controls (FPS, strategy)
- Console → controller-friendly design

4. Input System

- Touch → direct interaction
- Mouse/keyboard → precision
- Controller → smooth movement, less precision aiming

5. Setting & Theme

- Fantasy → RPG, adventure
- Sci-fi → flexible (action, strategy, racing)
- Horror → often action/survival
- Open world → supports exploration genres

6. World Design

- Open world → exploration, RPG
- Linear/tight → action-focused

7. Precedent (Existing Games)

- Compare with similar games
- Genres often evolve from existing patterns

Key Insight on Classification

- No single feature defines a genre
- Genre = **pattern recognition across multiple factors**

Combining Genres (Design Approach)

To create hybrid genres:

- Combine **mechanics**, Combine **systems**, Combine **player experiences**

Consistency matters: Features must be **integrated**, not randomly added

8. MDA Framework (Foundation for Genre Design)

Game = **3 components**:

1. **Mechanics** → rules, systems
2. **Dynamics** → player interaction with systems
3. **Aesthetics** → player experience/feeling

9. Game Aesthetics (8 Types)

These define **what the player feels**:

- | | |
|--|---|
| 1. Sensation → visuals, sound, feedback | 5. Fellowship → social interaction |
| 2. Fantasy → immersion in another world | 6. Discovery → exploration |
| 3. Narrative → story | 7. Expression → customization, identity |
| 4. Challenge → difficulty, obstacles | 8. Submission/Abnegation → time-pass, relaxation |

10. Why Aesthetics Matter for Genre

- Genres are not just mechanics, They are **experiential patterns**

Example:

- RPG → fantasy + narrative + expression,
- Casual → submission + simple challenge
- MMO → fellowship + progression

11. Case Insight (Subnautica Example)

Demonstrates:

- Multiple aesthetics working together:
 - Sensation (environment)
 - Fantasy (alien planet)
 - Narrative (story)
 - Challenge (survival)
 - Discovery (exploration)
 - Expression (base/vehicle customization)
 - Submission (replayability)
- **Key takeaway:** Strong games layer multiple aesthetics intentionally

12. Core Takeaways (What to Remember)

- Genres are **fluid, not fixed**
- Most games are **hybrids**
- Classification requires **multiple perspectives**
- Mechanics alone don't define genre — **experience does**
- Player type + function + content → core base
- Supporting factors refine classification
- Aesthetics = **what players actually feel**
- Good design = **intentional combination**, not random mixing

13. Practical Thinking Frame

When analyzing or designing a game:

- What **core genres** does it belong to?
- What **player experience (aesthetics)** is dominant?
- What **mechanics support that experience**?
- What **player types** does it attract?
- What **existing games** does it resemble?
- What **hybrid combination** is being used?

GAME DESIGN - LECTURE 03

1. Core Principle: Planning & Risk Awareness

- Game development is uncertain → planning reduces failure.
- Every game has **unique risks** (mechanics, visuals, audience fit, etc.).
- You must **identify + communicate risks**, not hide them.
- Investors expect awareness + mitigation strategies, not perfection.

2. Design Artifacts (Key Documents)

2.1 Concept Pitch (1–3 pages)

- Purpose: Sell the idea to investors/stakeholders.
- Must include:
 - Genre
 - Target audience
 - Game description
 - Unique/compelling features
 - Market context
 - Estimated cost & timeline
- Focus: **Why this game is worth the risk**

2.2 Game Design Document (GDD)

- Length: ~15–50 pages, constantly updated
- Purpose: Align design team, avoid unnecessary changes
- Contains:
 - Core mechanics
 - Features, Target audience
 - Game style/vision
- Role: **Non-technical blueprint of the game**

2.3 Technical Design Document (TDD)

- Length: 20–1000+ pages
- Purpose: Guide developers in execution
- Contains:
 - Tech stack (engine, APIs, languages)
 - Architecture, pseudocode, assets
 - AI, UI, gameplay systems
 - Risks, deadlines, milestones
- Role: **Technical execution + coordination tool**

3. Rapid Planning Tools

3.1 Storyboarding

- Quick, low-cost level/story planning, Visual sequence of scenes (like a comic strip)
 - Includes: Player actions, Scene context, Sound/music
- Used to test **flow + player interaction early**

3.2 Mood Boards

- Visual collage (colors, references, style)
- Helps define **look and emotional tone**
- Iterative → multiple versions required

3.3 Concept Sketches

- Early drawings of characters/environments
- Used as references → evolve heavily over time
- Key idea: **Don't get attached to early designs**

4. MDA Framework (Core Design Model)

4.1 Mechanics (Rules Layer)

- Rules, systems, actions, controls

- Example: timers, abilities, constraints

4.2 Dynamics (System Behavior)

- Emerges from mechanics interacting
 - Examples: Time pressure, Teamwork, Multiple playstyles, Atmosphere

4.3 Aesthetics (Player Experience)

- What players feel: Challenge, Fantasy, Narrative, Fellowship, Expression, Discovery, Sensation

Key Insight

- Designer flow: **Mechanics → Dynamics → Aesthetics**
- Player perception: **Aesthetics → Dynamics → Mechanics**

5. Mechanics → Dynamics Examples

- Countdown timer → Time pressure
- Complementary abilities → Teamwork
- Class/weapon variety → Multiple playstyles
- Lighting + AI + environment → Atmosphere

6. Dynamics → Aesthetics Mapping

- Time pressure → Challenge
- Teamwork → Fellowship
- Playstyles → Expression
- Visual/audio design → Sensation

7. Player-Centric Design

- Focus: Design with player involvement

- Process:

- Identify player needs
- Define requirements
- Build prototype
- Test with users
- Refine → repeat

Important Methods

- Playtesting (most critical)
- Focus groups
- Paper prototyping
- Ethnographic/cultural checks

Key Rule: Ask **specific questions**, not vague ones

8. Game Design Stages

8.1 Concept Stage

- Create idea + pitch document
- Build quick prototypes
- Validate assumptions
- Don't get attached → ideas may be discarded

8.2 Elaboration Stage

- Full development begins
- Iterative cycles (prototype → test → refine)
- Expect problems + surprises
- External testers help reveal blind spots

8.3 Tuning Stage

- Final polishing phase
- Remove unnecessary/low-value elements
- Heavy playtesting + bug fixing
- Takes **~30–50% of total development time**

9. Iteration & Reality of Design

- Game design is **non-linear and iterative**
- Reuse previous systems where possible
- External feedback is essential (bias exists internally)
- More perspectives → fewer blind spots

10. Key Takeaways (All Things to Remember)

- Planning is mandatory; unstructured ideas rarely succeed
- Every decision carries risk → identify and mitigate early
- Separate design (GDD) from implementation (TDD)
- Use quick tools (storyboards, mood boards) before heavy work

- MDA is the backbone: mechanics create experience indirectly
- Player perspective ≠ designer perspective
- Test constantly; assumptions are usually wrong
- Don't get attached to ideas → iteration will break them
- Most time is spent polishing, not creating
- Involve players and others early → reduces costly mistakes

GAME DESIGN - LECTURE 04

1. Core Principle: Game Development is Multi-Disciplinary

- Game development = **integration of multiple industries** (art, tech, business, design).
- No single individual can realistically handle full-scale modern game production.
- The “solo developer” idea is possible but **impractical for quality + scale**.
- Success depends on **coordinated teamwork across specialized roles**.

2. Industry Structure & Team Composition

- Large studios (AAA): ~25% programmers, ~75% non-technical support disciplines
- Key disciplines include:
 - Design, Art, Writing
 - Audio, Music
 - Programming, Engineering
 - Playtesting, UX/HCI
 - Marketing, Legal, Finance
 - Management, HR
- Teams scale based on **game complexity and market scope**

3. Team Classification

3.1 Non-Technical Teams (Majority)

Game Designer

- Defines gameplay, systems, story, progression
- Drives creative direction

Producer

- Ensures deadlines, budget, and scope control
- Prevents excessive design changes

Artists

- Create assets: characters, environments, animations
- Work through pipeline:
 - concept → modeling → texturing → animation

Managers

- Handle planning, resources, coordination
- Connect teams with business, legal, marketing

3.2 Technical Teams (Execution Layer)

Programmers

- Implement game systems, mechanics, tools
- Work inside game engine

Level Designers/Editors

- Build playable environments
- Balance gameplay using testing + UX input

Audio Engineers

- Handle music, sound effects, dialogue systems
- Must consider licensing/copyright

Playtesters

- Validate usability, bugs, player experience
- Critical for iterative improvement

4. Reality of Team Scale

- Even “simple” games require **multiple roles + coordination**
- Large titles involve **hundreds of contributors across countries**
 - Additional roles may include: Localization (translators), Cultural adaptation
- Complexity grows with **realism, simulation, and audience scale**

5. Inter-Team Communication

5.1 Managers ↔ Teams

- Ensure resources, timelines, quality, Conduct regular progress tracking meetings

5.2 Artists ↔ Programmers

- Critical dependency
- Assets must meet technical constraints (size, format, performance)
- Poor communication → rework, delays, or asset rejection

5.3 Team ↔ Client/Publisher

- Regular structured meetings
- Communication must be: Clear, Non-technical when needed, Respectful
- Late design changes from clients should be controlled

6. Example Insight: Small Game Complexity

Even a simple concept (e.g., object physics simulation) requires:

- Designers → define behavior
- Artists → create models + textures
- Programmers → physics, UI, controls
- Audio → feedback sounds
- Testing → realism + usability
- High realism → increased computation + development time

Key Insight: “Simple idea” ≠ “Simple execution”

7. Intra-Team Communication (Within Teams)

7.1 Regular Meetings

- Each member reports: Work completed, Issues faced, Ideas/improvements
- Followed by discussion/questions

7.2 Work Planning Behavior

- Members define next goals
- Slight overestimation helps avoid idle slowdown
- Transparency prevents bottlenecks

7.3 Communication Rules

- One speaker at a time
- Structured discussion
- Issues raised early

8. Scope Control & Prioritization

- Most game ideas are **too large for available time/resources**
- Must define:
 - Core features (must keep)
 - Secondary features (can be removed)
- Prefer **removing weak features over adding new ones**

9. Project Management Tools

9.1 Job Board System

- Tasks listed as small cards (name, description, deadline)
- Team members pick tasks independently
- Rules:
 - Finish before taking new work
 - Avoid dropping tasks
 - If dropped → update progress and return task

Impact:

- Reduces management overhead
- Encourages ownership and autonomy

9.2 Burndown Chart

- Tracks remaining work vs time
- Shows team velocity (speed of progress)
- Effects: Improves visibility, Boosts motivation, Highlights delays early

10. Communication System Summary

- Cross-team communication prevents **technical mismatches**
- Internal meetings maintain **alignment and accountability**
- Client communication ensures **business alignment**
- Poor communication = **time loss, rework, budget issues**

11. Team Dynamics & Mindset

- Teams are intentionally diverse → better creativity
- You will rarely work with like-minded people
- Collaboration quality impacts final product more than individual skill

12. Career Positioning Insight

- You should identify:
 - Your preferred role (design, dev, art, etc.)
 - Your strengths and comfort areas
- Understanding roles helps in:
 - Better learning direction
 - Stronger interviews
 - Clear career path

13. Key Takeaways (All Things to Remember)

- Game development is inherently collaborative; solo work doesn't scale
- Majority of work is non-programming (art, design, production, etc.)
- Clear communication between teams is critical to avoid costly mistakes
- Technical constraints must guide creative decisions
- Even small features can require complex coordination
- Regular structured communication keeps teams aligned
- Scope control is essential; cutting features is often necessary
- Task systems (job boards) improve autonomy and efficiency
- Progress tracking (burndown charts) improves morale and accountability
- Diverse teams produce better outcomes than uniform thinking
- Knowing your role in the system is key to long-term growth

While the previous UX design chapter also involved practical learning, this game design chapter leans even more heavily into hands-on work and application. Because of this stronger shift toward practical exploration, I've decided to keep the lecture notes for this particular chapter in markdown format instead of converting them into PDFs.

The lecture notes are available here:

[Github > Runarok > ux-game-intern > game-design > my-notes > lectures](#)

As a result, I will not be continuing the PDF versions from Lecture 05 onward.